

Layered Agent Fund

A layered LLM architecture for macro-rates research: isolated analyst agents → a reconciling portfolio-manager agent

Project write-up — data → analyst layer → PM layer

Universe: U.S. macro drivers & the nominal Treasury curve

Date: July 22, 2026

One line. *Two LLM layers turn seven isolated macro reads and a reconciling portfolio-manager read into a Treasury trade, and every number they produce is graded against a free, non-LLM baseline on the driver's own non-overlapping release clock.*

Table of Contents

Table of Contents	2
Summary	3
Sensibility	4
Data — What Game Are We Playing	5
Frequency and trading schedule	5
Model Construction	6
The no-look-ahead gate	6
The analyst layer	6
The PM layer	7
Parameter Search	8
Returns / Testing	9
Is the driver predictable at all? (free, no model)	9
Analyst layer — model capability sweep, inflation	9
PM layer — duration pod, 120 monthly meetings, 2016–2025	9
Read this honestly	9
What We Learned	11
On the design	11
On operating an LLM pipeline honestly	11
What we would fix next	11

Summary

We built a layered multi-agent system that turns raw macro data and FOMC text into views and trades on the U.S. rates complex, without ever handing a language model a finished answer to agree with. Seven isolated LLM “analysts,” each responsible for exactly one macro driver — inflation, labor tightness, curve slope, term premium, balance-sheet policy, financial conditions, inflation expectations — read only engineered measurements and their own driver’s policy language, and write a structured report: a direction, a conviction, a falsifier, and what evidence they were never given. A second LLM layer, the portfolio manager (PM), reads all seven reports each month and reconciles them into a per-driver view and, for two pods so far, a real two-leg Treasury trade.

Every number the system produces is graded against a free, non-LLM baseline — a persistence/mean-reversion feature, a mechanical consensus rule — on the driver’s own non-overlapping release clock, so “did the agent help” is always answered relative to what arithmetic alone would have gotten, not against a coin flip. The pipeline runs entirely offline against vendored FRED and FOMC data; only the scored model calls cost anything (roughly \$1–\$13 for a full 2005–2026 single-driver run depending on model, and \$5–\$6 for a 120-meeting PM run).

The headline results: a strong model (Opus) clears the best lookahead-free feature on inflation in-sample; two prompt-contract fixes on the PM layer (which coordinate system a conviction is expressed in, and whether the PM remembers its own last position) turned “PM beats its own analysts” from 0 of 5 drivers to 4 of 5, and turned the duration trade from statistically indistinguishable from zero into the first result that clears its mechanical baseline. Both the successes and the open risks — chiefly, whether historical-window LLM skill is real skill or training-data memorization — are reported with the same rigor.

Sensibility

The problem with how this is usually done. An earlier version of this pipeline made two mistakes that are easy to make by accident and hard to notice without measuring for them. First, the numeric arm handed the model its own finished answer — direction and conviction already computed — and asked it to “refine” it. Measured agreement between the model's output and the mechanical formula it was shown was 0.965: the model added nothing, because it was never asked to form a view, only to bless one. Second, the text arm sent every one of the seven analysts the identical FOMC statement. The only thing that varied across seven prompts was the driver's name on line one. Average pairwise correlation across the seven analysts' views rose from 0.221 to 0.339, and two analysts' views tracked another driver's evidence more closely than their own — instructions to “stay on your own driver” could not overcome shared input.

The fix, and why it is structural rather than a prompting trick. Both failures came from what crossed the first boundary, so that is what changed. Each analyst now receives exactly two channels about exactly one driver: engineered measurements from a *closed* vocabulary of operations (levels, changes, spreads, moving averages, distance from a stated policy constant) that cannot fit a parameter, standardize on the full sample, or score a direction — so a feature literally cannot express a forecast — and driver-partitioned FOMC language, diffed against the previous statement so the model sees what changed rather than four hundred words of boilerplate. No date ever appears in a prompt. A single shared gate (one dataclass, one pandas slice, reused verbatim by the analyst engine and the PM's meeting board) enforces that nothing downstream can ever see data from after the moment it is reasoning about. The model supplies 100% of the judgment; the engineering supplies 100% of the isolation.

Why we believe this can work at all. Before spending anything on a model call, the free, feature-only check showed inflation has real, if modest, structure: a mean-reversion reading on its trailing 12-month range holds up at $IC \approx -0.23$ whether you look at the full 2005–2026 sample or restrict to pre-COVID history — implying an achievable IR near 0.8 from a single measurement. That is the bar: not “beat a coin flip,” but “beat the best stable measurement, using both the arithmetic and the policy language a human analyst would actually read.” A cheap model (Haiku) does not clear that bar. A strong model (Opus) does, in-sample, and — more convincingly than the raw score — you can watch *why*: as capability rises from Haiku to Opus, correlation with the single most informative base-effect feature moves from near zero to -0.42 in the economically correct direction, which is the fingerprint of genuine reasoning over the measurement, not pattern-matching to a label.

The honesty built into the story. An LLM's apparent skill on historical macro data is confounded with memorizing what actually happened, because the backtest window sits inside every model's training data. We do not treat this as solved: every headline result in this write-up is reported next to a lookahead-free, non-LLM ceiling (so “does the agent exceed what pure arithmetic could achieve” is checkable), a post-training-cutoff slice is pulled out separately wherever the sample allows it, and one open question — whether the PM's degradation in the most recent period is regime difficulty or a memorization leak — is logged as **unresolved** in our own decision record rather than argued away. That is the sensibility: give the model only what a human analyst would have had, let it do all the reasoning, and hold every number to a free baseline before trusting it.

Data — What Game Are We Playing

This is a macro-rates strategy, not an equities or cross-sectional strategy: seven “drivers” of the U.S. rate complex, each backed by one FRED series (or a small family of them) and read against FOMC communications. Every series is vendored as CSVs in the repository, so the full 2005–2026 history runs offline with no API key; only the scored LLM calls need credentials.

Driver	FRED series	Text role
Inflation	CPIAUCSL, PCEPILFE	Every FOMC meeting discusses it
Labor tightness	UNRATE	Labor-market language
Curve slope (2s10s)	DGS2, DGS10	Policy-path language
Term premium	DGS10 (proxy)	Long-end / supply language
Balance sheet	WALCL	QT / runoff language
Financial conditions	NFCI	Credit / conditions language
Inflation expectations	T10YIE	Expectations language

Table 1. The seven drivers and their data sources.

The text side is 343 point-in-time FOMC statements and minutes, release-dated so a document is only visible on or after the date it actually published, and partitioned per driver by keyword cue so each analyst reads only its own driver's sentences (plus a diff against the previous release, so the model sees the *edit*, not the boilerplate).

Frequency and trading schedule

- **Analysts are graded on their own driver's natural release clock** — roughly monthly (CPI, jobs) — or, for daily market series (2s10s, the 10-year, NFCI, breakevens), resampled to month-end specifically so consecutive observations do not overlap. Overlap would make the t-statistics dishonest; a non-overlapping clock is what lets a naive t-stat be trusted without a Newey-West correction.
- **The PM meets monthly, at month end.** It does not wait for every analyst to have a synchronized, fresh view — it reads an “as-of snap”: each driver's most recent view as of that date, however stale, with the staleness shown explicitly rather than hidden. The labor read can legitimately be three weeks old on the last day of the month; the PM is told so.
- **The trade layer rebalances monthly** in a two-instrument Treasury universe (2-year and 10-year on-the-run proxies via DGS2/DGS10). One pod (“duration”) takes a directional level view with both legs the same sign; a second (“curve”) takes a 2s10s slope view with the legs equal and opposite.
- **Breadth is genuinely small.** About 12 independent bets per year per driver. Under the fundamental law of active management ($IR \approx IC \cdot \sqrt{\text{breadth}}$), that means an IR of 1.0 needs an IC near 0.29 — a far higher bar than a daily cross-sectional book needs at IC 0.05. This is stated up front and used as the yardstick for every result in this write-up, not introduced after the fact to explain away a weak number.

Model Construction

The pipeline is three stages: data feeds → an analyst layer (seven specialists) → a PM layer (one reconciler per pod). A fund/risk layer that allocates capital across pods does not exist yet; the analyst and PM layers hand off at two typed contracts (*DriverView*, *StrategyTrade*) built for that future layer to consume.

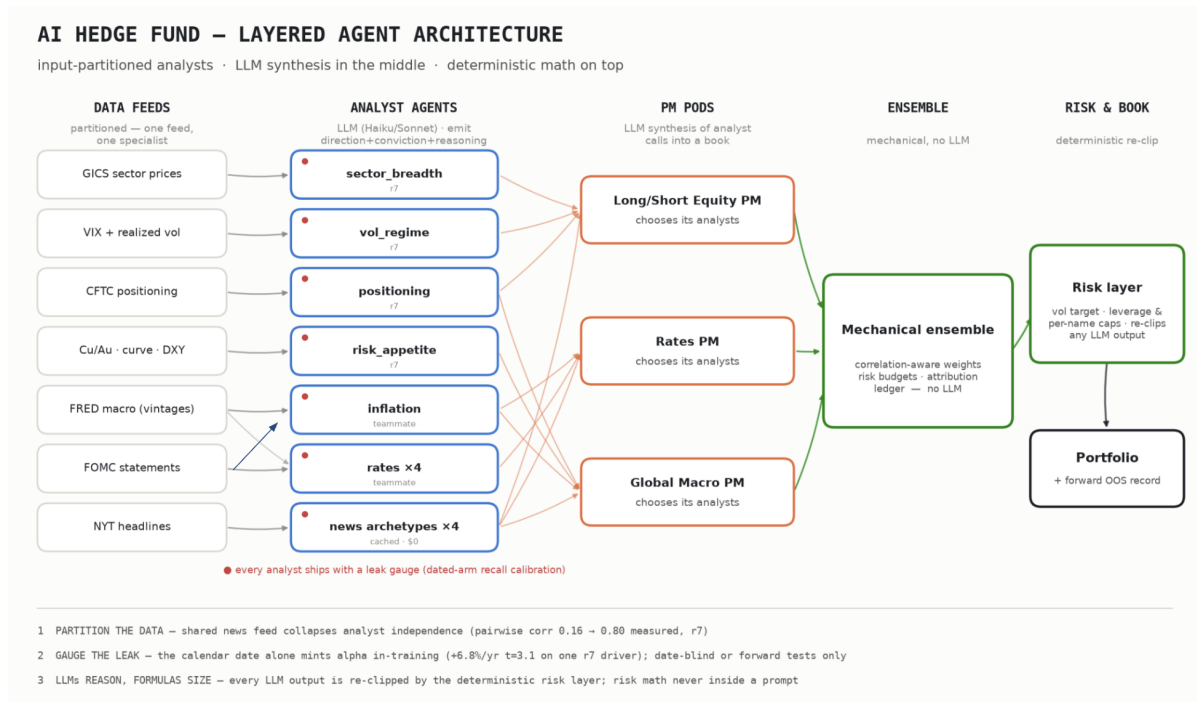


Figure 1. Data → analyst → PM → fund handoff.

The no-look-ahead gate

Every observable, at both layers, is read through one frozen, as-of view of the world: a slice of every series strictly $\leq$ the reasoning date. The analyst layer and the PM layer implement this with the identical one-line pandas idiom on purpose, so the two choke points in the codebase audit identically. Publication lag is corrected upstream (a monthly CPI print is shifted to its actual release date, not its reference date, before anything else touches it), so the gate protects the slice and the lag correction protects the timestamp — together, the end-to-end guarantee.

The analyst layer

- **Feature engine.** A closed vocabulary of twelve operations (level, diff, pct_change, year-over-year, moving average, spread, ratio, lag, distance-from-reference, rolling min/max, annualized change) — none of which can fit a parameter to outcomes or standardize on the full sample — computes each driver's measurements from a YAML persona file. Adding a driver is a config file, not a code change.
- **Text selection.** A cue-based selector compiles each driver's keywords to boundary-safe regex (plain substring matching once routed the Fed-funds target into the inflation analyst via the “2 percent” cue) and returns what changed, what dropped, and what recurring context survived since the previous FOMC document.

- **The analyst itself.** One specialist per driver, forced tool-call structured output (direction, conviction, a 120–250 word report, cited evidence, a falsifier, and what it was never given), reasoning over nothing but its own two channels. A failed call abstains (a flagged, zero-conviction “degraded” view) rather than falling back to a benchmark’s answer, so a scoring comparison is never contaminated by its own control.

The PM layer

- **The meeting.** A board replays every analyst’s saved views and, for a given date, hands the PM the most recent view per driver — present, stale, or absent — through the same as-of gate the analysts use.
- **The PM (LLM).** Reads the whole panel plus what each analyst says it was never given (a dependency graph only the PM can see, since each analyst is isolated by design), and reconciles into a per-driver conviction, prose, which analysts it leaned on or discounted, and — where the pod declares a trade mandate — a two-leg Treasury position with an explicit same-sign or opposite-sign constraint.
- **The mechanical PM.** A zero-cost, fully deterministic control: each driver’s conviction is half its own analyst and half the polarity-oriented panel mean; the trade is the panel’s rate-axis projection scaled onto the pod’s canonical legs. Every LLM PM result in this write-up is reported next to this baseline, not in isolation.

Parameter Search

There is no numerical optimizer anywhere in this pipeline, by design: a fitted parameter is exactly what the closed feature vocabulary exists to forbid, since fitting to outcomes would turn a measurement into a smuggled forecast. “Parameter search” here means a sequence of controlled, single-variable A/B arms, each checked against a pre-registered, non-LLM baseline rather than tuned to its own outcome.

Arm	What varied	Result
Feature richness	13 features (v1) → +6 base-effect / range features (v2)	Full-sample IC +0.122 → +0.156; pre-COVID IC +0.043 → +0.138
Model capability	Haiku 4.5 → Sonnet 5 → Opus 4.8, identical prompt	IC +0.187 → +0.340 → +0.492, monotonic; Opus holds pre-COVID (+0.447)
Feature description	Bare feature names vs. construction-only descriptions	Flat null on all three models — ruled out as a lever
Text partitioning	Whole-document control vs. driver-partitioned cue selection	Partitioning is what fixed the 0.221→0.339 correlation collapse
PM answer space	Driver-space vs. rate-space convictions, bound to one declared key	PM-vs-analyst win rate 0/5 → 4/5 drivers
PM memory	Stateless PM vs. shown its own prior arbitration	Sign flips 45.8% → 11.0%; trade t +0.08 → +1.73

Table 2. Each arm changes exactly one variable against a fixed baseline.

The two negative results (feature description, and — more subtly — the individual base-effect features once conviction was already anchored on momentum) mattered as much as the positive ones: they ruled out “better labels” as a cheap lever and showed that model capability, not feature presentation, was the binding constraint. Every declared quantity that could bias a result — each driver’s polarity sign, each pod’s leg-sign convention — is fixed in advance in the persona/pod configuration and never refit from an outcome, the same rule the feature vocabulary itself is held to.

Returns / Testing

Is the driver predictable at all? (free, no model)

Signal (headline CPI YoY)	Full-sample IC (t)	Pre-COVID IC (t)	Reading
12-month-high mean reversion	−0.182 (−2.94)	−0.229 (−3.12)	Stable — the bar
3-month acceleration (momentum)	+0.158 (2.55)	+0.069 (0.92)	Collapses out of sample
12-month-low mean reversion	−0.193 (−3.13)	−0.118 (−1.58)	Collapses out of sample

Table 3. The stable effect implies $IR \approx 0.23 \times \sqrt{11.8} \approx 0.79$ from a single measurement — real, achievable with zero model spend, and the bar every LLM result below is measured against.

Analyst layer — model capability sweep, inflation

Model	IC (full)	t	Pre-COVID IC	Post-cutoff IC (n=15)
Haiku 4.5	0.187	3.0	0.142	0.162
Sonnet 5	0.340	5.7	0.253	0.163
Opus 4.8	0.492	9.0	0.447	0.181

Table 4. IC rises monotonically with capability and the strongest model holds pre-COVID ($t \approx 6$, $n=179$) — evidence the design was reasoner-limited, not architecture-limited. The post-cutoff slice ($n=15$, too small to be conclusive) shows the model spread collapsing to -0.16 – 0.18 , which we report as a caution on taking the in-sample number at face value, not as a refutation.

PM layer — duration pod, 120 monthly meetings, 2016–2025

Arm	PM beats analyst (drivers)	Sign flips m/m	Trade P&L t-stat	vs. mechanical
Before fixes (OFF)	0 / 5	45.8%	+0.08	t +0.26 (rule wins)
After fixes (ON)	4 / 5	11.0%	+1.73	clears the rule
Mechanical baseline	— (by construction)	33.6%	+0.26	—

Table 5. “Before/after” = two prompt-contract fixes (answer-space + memory), entangled in this run — not yet a clean factorial. Balance-sheet IC alone recovered from -0.167 to $+0.713$ essentially matching its own analyst.

Read this honestly

- Cost: a full 2005–2026 single-driver analyst run is ~\$1 (Haiku) to ~\$13 (Opus); the 120-meeting PM run is \$5.40–\$5.73.
- Turnover fell 86% once the PM had memory, so the 119 “observations” in the ON trade are carried by a handful of persistent positions — effective independence is lower than the raw count suggests, and $t=+1.73$ is read as suggestive, not decisive.
- The trade P&L is yield-space ($\sum \text{leg weight} \times \Delta \text{yield}$) with no duration weighting, no carry, and no transaction costs. It is the exact quantity the model was told it would be scored on — it is not a bond return and not a real P&L.
- **Unresolved, stated as such:** the PM's trade degrades sharply in the most recent period (hit rate $0.55 \rightarrow 0.38$). The team's own mechanical control initially looked like proof this was regime difficulty rather than memorization — then

caught itself: the mechanical PM is deterministic only in its arithmetic, and its inputs are still the LLM analysts' convictions, so a leak upstream would produce exactly this "they degrade together" pattern. The claim was withdrawn in the decision log rather than kept.

What We Learned

On the design

- **Isolation has to be structural, not instructional.** Telling seven analysts “reason about your own driver only” did nothing while they were reading the same document; only removing the shared text fixed the correlation collapse. A prompt instruction cannot override shared information sitting in the context.
- **Showing a model the answer teaches it to agree, not to reason.** 0.965 correlation with a mechanical formula, once the formula's output was in the prompt, is the sharpest single number in this project for why “refine a computed signal” is the wrong shape for an LLM analyst.
- **A model can be told a mechanism and still not act on it if the feature is bare.** The mandate said “weigh momentum and the level relative to target” from the start; the model still latched onto whichever measurement was most visually salient until a stronger model, not a better label, closed the gap. Money spent on more features pays off only once the model is capable enough to use the features it already has — confirmed directly by a null result on feature descriptions.

On operating an LLM pipeline honestly

- **LLM output fails in mechanical, catchable ways.** Models return a comma-joined string where an array was asked for, wrap JSON in prose, or occasionally write structured output into the wrong free-text field (measured at roughly one meeting in six for one such failure). Every one of these was caught by measuring actual failure rates and given a defensive parser — never silently ignored or averaged away.
- **A metric can be wrong about itself.** Three separate false signals in the automated report-quality checks — a “trade-naming” false positive on the word “position,” a “hallucination” false positive on legitimately citing the text channel, and a parsing bug that shredded a string into characters — were each caught and fixed before being trusted. Verifying a surprising number before believing it was as important as building the number in the first place.
- **Memorization does not disappear just because your control is “mechanical.”** If a deterministic rule is still built on top of LLM-produced inputs, it inherits any upstream leak, and “both arms degrade together” looks exactly like proof of innocence when it may be proof of a shared contamination. The fix requires a control with no LLM anywhere in the stack — logged as the next concrete step, not glossed over.

What we would fix next

- One pod-level mandate rule — that a trade's leg signs must respect the pod's declared convention (same-sign for a level bet, opposite-sign for a slope bet) — is currently measured after the fact but not enforced at the point the trade is parsed, so a mandate-violating trade can sit inside the P&L statistics indistinguishable from a valid one unless the violation rate is checked separately. Small fix, real gap in an otherwise carefully audited seam.
- The training-cutoff memorization question for the PM's trade is open and specified, not closed: it needs the lookahead-free feature/persistence baseline run for the five rates drivers this pod reads (done so far only for inflation), split pre/post cutoff, so the leak question can be answered without any LLM in the control.
- Publication timing is currently a fixed, declared lag per series rather than true vintage-archive data — the standard practical approximation, and the one place the no-look-ahead guarantee rests on an assumption rather than a mechanically enforced fact.

Sources: docs/analyst-layer.md, docs/pm-layer.md, docs/decisions.md, docs/leak-and-disagreement.md and the run files they cite (reports/inflation_ic*.jsonl, reports/sweep_*.jsonl, reports/pm/duration_*.jsonl). All figures above are taken directly from those

records.